

41-monitoring

41. Monitoring & Error Tracking

Level: □ Advanced **Jam:** 10 (5 minggu × 2 sesi)
Prasyarat: Node.js & Express (Modul 6), pernah deploy aplikasi **Output:** Express app with Sentry + structured logging + health dashboard

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Setup structured JSON logging dengan Pino & Winston
- Integrasi Sentry untuk error tracking & performance monitoring
- Optimasi error grouping dengan fingerprint & breadcrumbs
- Build health check endpoint (liveness + readiness)
- Implement graceful shutdown production-ready
- Konfigurasi uptime monitoring & alerting (email, Slack, Discord)
- Trace performance bottleneck dengan APM tools
- Menyusun incident response checklist untuk produksi

Materi

Sesi	Topik	File
1	Structured Logging — Pino vs Winston, log levels, request logging middleware, log rotation	01-structured-logging.md
2	Sentry Error Tracking — DSN setup, Express middleware, source maps, breadcrumbs, performance tracing	02-sentry-error-tracking.md

Sesi	Topik	File
3	Health Check & Monitoring — /healthz, /readyz, dependency checks, graceful shutdown, monitoring dashboard	03-health-check-monitoring.md
4	Production Monitoring — uptime monitoring, alerting, APM, incident response checklist	04-production-monitoring.md

Output Akhir Modul

Production-ready Express app with full observability stack — structured JSON logging (Pino), Sentry error tracking & performance monitoring, health check endpoints (liveness + readiness + dependency probes), graceful shutdown, uptime monitoring config, dan incident response run-book.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- “Generate Pino logger config with different log levels per environment”
- “Write Express request logging middleware that captures method, url, duration, and status code”
- “How do I configure Sentry source maps upload in my CI pipeline?”
- “Generate health check endpoints for Express with PostgreSQL and Redis ping”
- “Explain the difference between liveness and readiness probes in Kubernetes context”
- “Generate Slack webhook alert payload for 5xx error threshold breach”
- “Help me analyze this APM waterfall chart — where’s the bottleneck?”
- “Write an incident response checklist for a production API outage”

41.1 Structured Logging

Kenapa `console.log` Ga Cukup?

`console.log` works untuk development, tapi di production:

- **Ga terstruktur** — susah parsing, grep, atau split
- **Ga ada level** — error vs debug sama aja
- **Ga ada metadata** — timestamp, requestId, service name ilang
- **Ga bisa routing** — ke file, stdout, Elasticsearch? Manual semua
- **Kinerja** — `console.log` blocking kalo stdout lambat

```
// ❌ console.log — nyampah
```

```
[2025-03-15T10:30:00] User login success
```

```
[2025-03-15T10:30:01] Error: something happened
```

```
[2025-03-15T10:30:02] Query took 150ms
```

```
// ✅ JSON structured — bisa diprogram
```

```
{ "level": 30, "time": 1742013000000, "msg": "User login success", "userId": 42 }
```

```
{ "level": 50, "time": 1742013001000, "msg": "Error: something happened", "err": { "message": "something happened" } }
```

```
{ "level": 20, "time": 1742013002000, "msg": "Query took 150ms", "query": "SELECT ...", "duration": 150 }
```

Pino vs Winston

Fitur	Pino	Winston
Performance	❌ Tercepat (~2x Winston)	Cepat, tapi kalah dari Pino
Bundle size	Kecil (~100KB)	Lebih besar
JSON default	❌ Ya — output JSON native	❌ Default string, perlu format
Transports	Built-in (file, rotate, browser)	Transport system modular
Levels	fatal, error, warn, info, debug, trace	error, warn, info, http, verbose, debug, silly
Custom level	Lewat <code>customLevels</code>	native support
Piped protocol	❌ Bisa pipe ke external	❌ Ga ada
Redaction	❌ <code>pino.redact</code> built-in	Perlu library tambahan
Child loggers	❌ Native	❌ Native
Ecosystem	Cepet berkembang	Mature, banyak plugin

Rekomendasi: - **Pino** — buat performa tinggi, serverless, atau production skala - **Winston** — kalo butuh banyak transport custom atau udah legacy pake Winston

Log Levels

```
// Dari paling penting ke paling detail:  
// fatal → 60 – Aplikasi crash, ga bisa recover  
// error → 50 – Error yang perlu ditindaklanjuti  
// warn → 40 – Something unexpected, tapi ga fatal  
// info → 30 – Normal operation (user login, request sukses)  
// debug → 20 – Development info, matiin di production  
// trace → 10 – Sangat detail, tracing function masuk
```

Best practice level di production: info **Development / debugging:** debug

Setup Pino di Express

Install

```
npm install pino  
# Atau kalo mau pino-pretty buat development  
npm install -D pino-pretty
```

Logger Config

```
// src/lib/logger.ts  
import pino from 'pino';  
import path from 'path';  
  
const isProduction = process.env.NODE_ENV === 'production';  
  
const logger = pino({  
  level: process.env.LOG_LEVEL || (isProduction ? 'info' : 'debug'),  
  
  // Formatting prettier di development  
  ...(isProduction  
    ? {}  
    : {  
      transport: {  
        target: 'pino-pretty',  
        options: {  
          colorize: true,  
          translateTime: 'SYS:yyyy-mm-dd HH:MM:ss',  
          ignore: 'pid,hostname',  
        },  
      },  
    })  
});
```

```

        },
      },
    )),

    // Redact sensitive fields
    redact: {
      paths: ['req.headers.authorization', 'req.headers.cookie', 'password', 'secret'],
      censor: '[REDACTED]',
    },
  });

export default logger;

```

Pake di Express

```

// src/index.ts
import express from 'express';
import logger from '../lib/logger';

const app = express();
const PORT = process.env.PORT || 3000;

app.get('/api/users', async (req, res) => {
  logger.info({ userId: req.query.id }, 'Fetching user data');

  try {
    const user = await getUser(req.query.id);
    logger.debug({ user }, 'User fetched successfully');
    res.json(user);
  } catch (err) {
    logger.error({ err, userId: req.query.id }, 'Failed to fetch user');
    res.status(500).json({ error: 'Internal server error' });
  }
});

app.listen(PORT, () => {
  logger.info({ port: PORT }, 'Server started');
});

```

Request Logging Middleware

Biar tiap request tercatat otomatis — method, url, duration, status code.

```

// src/middleware/requestLogger.ts
import { Request, Response, NextFunction } from 'express';

```

```

import logger from '../lib/logger';

export function requestLogger(req: Request, res: Response, next: NextFunction) {
  const start = Date.now();
  const { method, url } = req;

  // Tangkap response selesai
  res.on('finish', () => {
    const duration = Date.now() - start;
    const { statusCode } = res;

    const level = statusCode >= 500 ? 'error'
      : statusCode >= 400 ? 'warn'
      : 'info';

    logger[level]({
      method,
      url,
      statusCode,
      duration: `${duration}ms`,
      contentType: res.get('content-length'),
      // Kalo pake correlation ID
      requestId: req.headers['x-request-id'],
    }, 'Request completed');
  });

  next();
}

// Pakai di app
app.use(express.json());
app.use(requestLogger);

// Output kira-kira:
// {"level":30,"time":....,"msg":"Request completed","method":"GET","url":"/api/u

```

Log Rotation

Log file di production bisa gede banget. Butuh rotation — putar log setiap hari / setiap sekian MB, simpan N backup, hapus yang lama.

Pake pino-roll

```

npm install pino-roll
// src/lib/logger.ts

```

```
import pino from 'pino';
import PINOROLL from 'pino-roll';

const logger = pino({
  level: 'info',
}, PINOROLL.createStream({
  file: path.join(__dirname, '../logs/app.log'),
  frequency: 'daily', // Rotasi setiap hari
  maxFiles: 14, // Simpan 14 hari terakhir
  size: '10m', // Atau rotasi kalo udah 10MB
}));
```

Alternatif: Eksternal Log Rotation

Kalo pake PM2 / systemd, bisa pake logrotate di Linux:

```
# /etc/logrotate.d/myapp
/path/to/app/logs/*.log {
  daily
  rotate 14
  compress
  delaycompress
  missingok
  notifempty
  copytruncate
}
```

Child Logger — Context Per Request

Biar log terikat ke satu request, pake child logger dengan correlation ID.

```
// src/middleware/attachLogger.ts
import { Request, Response, NextFunction } from 'express';
import logger from '../lib/logger';
import { v4 as uuidv4 } from 'uuid';

export function attachLogger(req: Request, res: Response, next: NextFunction) {
  const requestId = req.headers['x-request-id'] || uuidv4();

  // Child logger - semua log dari request ini punya requestId
  req.log = logger.child({ requestId });

  // Kirim balik header
  res.setHeader('x-request-id', requestId);
}
```

```

    next();
  }
  // Pake di route
  app.get('/api/orders/:id', (req, res) => {
    req.log.info({ orderId: req.params.id }, 'Fetching order');
    // Output: {"level":30,"requestId":"abc-123","orderId":"42","msg":"Fetching or
  });

```

JSON Logging — Kenapa Penting

Grepping manual vs tooling

```

// Tanpa JSON – susah filtering
grep "error" app.log | grep "user-42"

// JSON + jq – query powerful
cat app.log | jq 'select(.level >= 50) | {msg, userId, timestamp}'
cat app.log | jq 'select(.duration > 1000) | .url'

```

Integrasi dengan log management

JSON structure bikin integrasi ke ELK (Elasticsearch, Logstash, Kibana), Datadog, atau Grafana Loki jadi **drop-in** — tinggal kirim log stream, mereka parse otomatis.

// Standard format yang dikenali banyak tools

```

{
  "level": 30,
  "time": 1742013000000,
  "pid": 1234,
  "hostname": "server-01",
  "requestId": "abc-123",
  "method": "GET",
  "url": "/api/users",
  "statusCode": 200,
  "duration": "12ms",
  "msg": "Request completed"
}

```

Latihan

1. Setup Pino logger dengan konfigurasi: level debug di development, info di production. Implementasikan pino-pretty untuk dev dan redact untuk field password, token, secret. Tulis kode setup + contoh 3 log dengan level berbeda.

2. Buat Express middleware requestLogger yang mencatat: method, url, duration (ms), status code, dan content-length. Gunakan `res.on('finish')`. Log level: error untuk ≥ 500 , warn untuk ≥ 400 , info untuk sisanya.
3. Implementasikan child logger per-request dengan correlation ID (UUID). Tampilkan bagaimana log dari route `/api/users/:id` dan `/api/orders` memiliki requestId yang berbeda tapi konsisten dalam satu request. Tulis kode middleware + contoh output JSON.
4. Buat script log rotation sederhana: simpan log ke `logs/app.log`, rotasi harian, simpan 7 hari terakhir. Pake `pino-roll` atau `logrotate` config. Tulis konfigurasi lengkap dan cara testing rotasi-nya.

41.2 Sentry Error Tracking

Kenapa Sentry?

Error logging ke file aja ga cukup buat production:

- **Realtime notification** — tau error sebelum user komplek
- **Error grouping** — ribuan error yang sama $\Rightarrow$ 1 issue, bukan spam
- **Context kaya** — stack trace, breadcrumbs, user, environment, browser
- **Source maps** — error dari minified code balik ke source asli
- **Performance tracing** — liat endpoint mana yang lambat
- **Release tracking** — tau error muncul di deploy mana
- **Breadcrumbs** — jejak aksi user sebelum error terjadi

Setup Sentry di Express

1. Daftar & Dapatkan DSN

Buat akun di sentry.io → New Project → Express/Node.js → copy DSN.

DSN format: `https://<key>@<org>.ingest.sentry.io/<project>`

2. Install

```
npm install @sentry/node
```

```
npm install -D @sentry/webpack-plugin # buat source maps upload
```

3. Init di Entry Point

PENTING: Inisialisasi Sentry **sebelum** middleware/router lain.

```
// src/index.ts – PALING ATAS
import * as Sentry from '@sentry/node';
import { ProfilingIntegration } from '@sentry/profiling-node';

Sentry.init({
  dsn: process.env.SENTRY_DSN,
  environment: process.env.NODE_ENV || 'development',
  release: process.env.RELEASE_VERSION || '1.0.0',

  // Set tracesSampleRate – 1.0 = 100% request, 0.1 = 10%
  tracesSampleRate: process.env.NODE_ENV === 'production' ? 0.1 : 1.0,

  // Profiling – optional
  profilesSampleRate: process.env.NODE_ENV === 'production' ? 0.1 : 1.0,
  integrations: [new ProfilingIntegration()],

  // Attach stack trace ke semua error
  attachStacktrace: true,

  // Max breadcrumbs
  maxBreadcrumbs: 50,
});

// Express setup
import express from 'express';
const app = express();

// Request handler – tambahkan request data ke event
app.use(Sentry.Handlers.requestHandler());

// Tracing handler – kalo pake performance monitoring
app.use(Sentry.Handlers.tracingHandler());

// Routes...
app.get('/api/users', async (req, res) => {
  // ...
});

// Error handler – WAJIB: Sentry error handler (taruh setelah routes)
app.use(Sentry.Handlers.errorHandler({
  shouldHandleError(error) {
    // Kirim ke Sentry cuma kalo 5xx
  }
}));
```

```

    return error.status >= 500 || error.status === undefined;
  },
}));

// Fallback error handler
app.use((err, req, res, next) => {
  res.status(err.status || 500).json({
    error: 'Internal Server Error',
  });
});

```

4. Manual Error Reporting

```

import * as Sentry from '@sentry/node';

// Capture exception – otomatis ambil stack trace
try {
  await processPayment(orderId);
} catch (err) {
  Sentry.captureException(err, {
    tags: { orderId, paymentGateway: 'stripe' },
    level: 'error',
  });
}

// Capture message – buat log penting
Sentry.captureMessage('Payment gateway timeout', {
  level: 'warning',
  tags: { gateway: 'stripe' },
  extra: { orderId, timeout: 5000 },
});

// Set user context – biar tau user mana yang kena error
Sentry.setUser({ id: req.user.id, email: req.user.email });

```

Error Grouping & Fingerprint

Sentry group error otomatis berdasarkan stack trace. Tapi kadang satu error dikelompokin beda atau error beda dikelompokin sama.

Custom Fingerprint

```

Sentry.captureException(new Error('Database connection failed'), {
  // Paksa jadi satu grup untuk error database
  fingerprint: ['database-connection-error', process.env.DB_HOST],
});

```

```

});

// Atau bedain berdasarkan error code
app.use((err, req, res, next) => {
  if (err.code === 'ECONNREFUSED') {
    Sentry.setContext('Database connection refused. Check DB_HOST:', {
      dbHost: process.env.DB_HOST,
      dbPort: process.env.DB_PORT,
      service: 'payment-service',
    });
  }

  Sentry.captureException(err, {
    fingerprint: [err.code || err.name, req.path],
  });

  next(err);
});

```

Best Practice Fingerprint

Situasi	Fingerprint Strategy
Error external API timeout	['external-api-timeout', apiName]
Database connection error	['db-error', dbType]
Validation error	['validation', fieldName]
404 route	['not-found', requestPath] — jangan per-path
Rate limit	['rate-limit'] — 1 grup aja

Breadcrumbs

Breadcrumbs = trail aksi user sebelum error. Sentry otomatis record: HTTP request, console.log, navigation, dll. Tapi kita bisa add manual.

```

import * as Sentry from '@sentry/node';

// Otomatis – Sentry record request handler
app.get('/api/checkout', async (req, res) => {
  // Manual breadcrumb
  Sentry.addBreadcrumb({
    category: 'checkout',
    message: 'Starting checkout process',
    level: 'info',
    data: { cartItems: cart.length, total: cartTotal },
  });
});

```

```

try {
  await createOrder(cart);
  Sentry.addBreadcrumb({
    category: 'checkout',
    message: 'Order created',
    level: 'info',
    data: { orderId: newOrder.id },
  });

  await chargePayment(newOrder.id);
} catch (err) {
  Sentry.addBreadcrumb({
    category: 'checkout',
    message: 'Payment failed',
    level: 'error',
    data: { orderId: newOrder?.id, errorMessage: err.message },
  });

  Sentry.captureException(err);
}
});

```

Breadcrumb levels: debug, info, warning, error

Batasan: Default max 100 breadcrumbs. Atur lewat maxBreadcrumbs: 50 di init.

Performance Tracing

Transactions & Spans

Transaction = 1 request (misal GET /api/users). Spans = sub-operasi dalam request (query DB, call external API).

```

app.get('/api/orders/:id', async (req, res) => {
  // Transaction otomatis dari tracingHandler, tapi bisa manual:
  const transaction = Sentry.startTransaction({
    op: 'http.server',
    name: `GET /api/orders/:id`,
  });

  // Set transaction aktif
  Sentry.getCurrentHub().configureScope(scope => scope.setSpan(transaction));

  try {
    // Span 1 – database query

```

```

const orderSpan = transaction.startChild({
  op: 'db.query',
  description: 'SELECT orders WHERE id = ?',
});
const order = await db.query('SELECT * FROM orders WHERE id = ?', [req.params.id]);
orderSpan.finish();

// Span 2 – external API call
const paymentSpan = transaction.startChild({
  op: 'http.client',
  description: 'GET payment-service/status',
});
await fetch(`https://payment-api/status/${order.paymentId}`);
paymentSpan.finish();

res.json(order);
} catch (err) {
  Sentry.captureException(err);
  res.status(500).json({ error: 'Failed' });
} finally {
  // WAJIB: finish transaction
  transaction.finish();
}
});

```

Melihat Performance di Sentry

Di Sentry dashboard → Performance → liat:

- **Response time distribution** — historis, spike detection
- **Waterfall chart** — breakdown tiap span
- **Slowest endpoints** — top N endpoint paling lambat
- **Transaction summary** — throughput, p50, p95, p99

Automatic Instrumentation

Sentry otomatis nge-instrument:

- Express route handler (udah jadi transaction)
- Database queries (jika pake ORM yang didukung: Prisma, Sequelize, Mongoose)
- Outgoing HTTP requests (fetch, axios, got)
- `console.log` → jadi breadcrumbs

Tinggal pastiin `tracingHandler()` dipasang.

Source Maps

Kode production biasanya di-minify/transpile. Stack trace di Sentry jadi:

```
Error: Something broke
  at o (bundle.js:1:12345)
  at n (bundle.js:1:67890)
```

Source maps balikin ke source asli:

```
Error: Something broke
  at processPayment (src/services/payment.ts:42:10)
  at handler (src/routes/checkout.ts:15:3)
```

Upload Source Maps via CI

```
npm install @sentry/cli
```

```
# Set env
```

```
export SENTRY_AUTH_TOKEN=your_auth_token
```

```
export SENTRY_ORG=your_org
```

```
export SENTRY_PROJECT=your_project
```

```
# Create release & upload
```

```
sentry-cli releases new "$RELEASE_VERSION"
```

```
sentry-cli releases files "$RELEASE_VERSION" upload-sourcemaps ./dist --url-prefix
```

```
sentry-cli releases set-commits "$RELEASE_VERSION" --auto
```

```
sentry-cli releases finalize "$RELEASE_VERSION"
```

JANGAN upload source maps ke production public — biar cuma Sentry yang pake.

Otomatis dengan Webpack/Vite

```
// sentry.webpack.config.js
```

```
const { sentryWebpackPlugin } = require('@sentry/webpack-plugin');
```

```
module.exports = {
```

```
  // ...
```

```
  plugins: [
```

```
    sentryWebpackPlugin({
```

```
      org: 'my-org',
```

```
      project: 'my-project',
```

```
      authToken: process.env.SENTRY_AUTH_TOKEN,
```

```
      release: { name: process.env.RELEASE_VERSION },
```

```
    }),
```

```
  ],  
};
```

Release Tracking & Deploy Notification

Kasih tau Sentry tiap deploy baru — biar bisa liat error muncul di release mana.

```
# Di CI/CD pipeline (GitHub Actions, GitLab CI, dll)  
sentry-cli releases new "$CI_COMMIT_TAG"  
sentry-cli releases set-commits "$CI_COMMIT_TAG" --auto  
sentry-cli releases finalize "$CI_COMMIT_TAG"  
sentry-cli releases deploys "$CI_COMMIT_TAG" new -e production
```

Integrasi dengan Git

Sentry connect ke GitHub/GitLab — liat suspect commit yang introduce error langsung di dashboard.

Deploy Notifications

Di Sentry → Alerts → set aturan:

- **When:** Error rate > 10% dalam 5 menit
- **If:** Release baru
- **Then:** Kirim email, Slack, PagerDuty, dll.

Latihan

1. Setup Sentry di Express app: inialisasi dengan DSN, pasang requestHandler, tracingHandler, dan errorHandler. Konfigurasi: `tracesSampleRate: 0.2` di production, active `attachStack-trace`. Tulis kode lengkap dari init sampai error handler.
2. Implementasikan custom fingerprint untuk: (a) database timeout error, (b) validation error per field, (c) 404 not found. Jelaskan strategi fingerprint untuk masing-masing dan tulis kode `captureException` dengan fingerprint yang sesuai.
3. Buat manual breadcrumbs di flow checkout: cart validation → create order → charge payment. Setiap step punya breadcrumbs sendiri. Kalo payment gagal, tambah breadcrumb error + `captureException`. Tulis kode lengkap.
4. Setup performance tracing: buat transaction manual untuk route `POST /api/orders` yang punya 3 spans — validasi input (db query), create order (db write), kirim email (external API

call). Finish transaction setelah selesai. Tampilkan kode + bagaimana cara liat waterfall chart di Sentry.

41.3 Health Check & Monitoring

Kenapa Health Check?

Production app musti siap mati kapan aja — restart, scaling, deploy. Health check ngasih tau orchestrator (Kubernetes, Docker, load balancer) kapan aplikasi siap nerima traffic.

Dua jenis probe:

Probe	Fungsi	Kalo Gagal
Liveness (/healthz)	Apakah app masih hidup? Ga hang?	Container di-restart
Readiness (/readyz)	Apakah app siap nerima traffic?	Traffic dialihkan

Contoh kasus:

- App jalan tapi DB mati → liveness OK (proses jalan), readiness FAIL (ga bisa serve request)
- App memory leak → liveness FAIL (app ga responsif), container direstart
- Cold start butuh 10 detik → readiness FAIL sampe app siap

Endpoint /healthz (Liveness)

Check simpel — apakah proses app masih jalan.

```
// src/routes/health.ts
import { Router, Request, Response } from 'express';

const router = Router();

// Liveness — cuma check app jalan
router.get('/healthz', (_req: Request, res: Response) => {
  res.status(200).json({
    status: 'ok',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
  });
});
```

```

export default router;

// src/index.ts
import healthRouter from './routes/health';
app.use(healthRouter);

```

Endpoint /readyz (Readiness)

Check dependensi benaran — DB, Redis, external API.

```

// src/routes/health.ts (lanjutan)
import { Router, Request, Response } from 'express';
import { Pool } from 'pg';
import { createClient } from 'redis';

const router = Router();

interface HealthCheck {
  name: string;
  status: 'ok' | 'error';
  error?: string;
  latency?: number;
}

// Readiness – check dependensi
router.get('/readyz', async (_req: Request, res: Response) => {
  const checks: HealthCheck[] = [];

  // Check PostgreSQL
  const pgPool = new Pool({ connectionString: process.env.DATABASE_URL });
  const pgStart = Date.now();
  try {
    await pgPool.query('SELECT 1');
    checks.push({ name: 'postgres', status: 'ok', latency: Date.now() - pgStart });
  } catch (err) {
    checks.push({
      name: 'postgres',
      status: 'error',
      error: err.message,
      latency: Date.now() - pgStart,
    });
  } finally {
    await pgPool.end();
  }
}

```

```

// Check Redis
if (process.env.REDIS_URL) {
  const redis = createClient({ url: process.env.REDIS_URL });
  const redisStart = Date.now();
  try {
    await redis.connect();
    await redis.ping();
    checks.push({ name: 'redis', status: 'ok', latency: Date.now() - redisStart });
  } catch (err) {
    checks.push({
      name: 'redis',
      status: 'error',
      error: err.message,
      latency: Date.now() - redisStart,
    });
  } finally {
    await redis.disconnect();
  }
}

// Check external API
if (process.env.EXTERNAL_API_URL) {
  const apiStart = Date.now();
  try {
    const response = await fetch(`${process.env.EXTERNAL_API_URL}/health`);
    if (response.ok) {
      checks.push({ name: 'external-api', status: 'ok', latency: Date.now() - apiStart });
    } else {
      checks.push({
        name: 'external-api',
        status: 'error',
        error: `HTTP ${response.status}`,
        latency: Date.now() - apiStart,
      });
    }
  } catch (err) {
    checks.push({
      name: 'external-api',
      status: 'error',
      error: err.message,
      latency: Date.now() - apiStart,
    });
  }
}

// Overall status

```

```

const allOk = checks.every(c => c.status === 'ok');
const statusCode = allOk ? 200 : 503;

res.status(statusCode).json({
  status: allOk ? 'ok' : 'degraded',
  timestamp: new Date().toISOString(),
  uptime: process.uptime(),
  checks,
});
});

export default router;

```

Dependency Check Pattern — Best Practices

Timeout per Check

Jangan biarin satu check lambat nahan yang lain.

```

async function checkWithTimeout<T>({
  name: string,
  fn: () => Promise<T>,
  timeoutMs = 3000
}): Promise<HealthCheck> {
  const start = Date.now();
  try {
    const result = await Promise.race([
      fn(),
      new Promise<never>((_, reject) =>
        setTimeout(() => reject(new Error('Timeout')), timeoutMs)
      ),
    ]);
    return { name, status: 'ok', latency: Date.now() - start };
  } catch (err) {
    return { name, status: 'error', error: err.message, latency: Date.now() - start };
  }
}

// Pake:
const pgCheck = await checkWithTimeout('postgres', () => pgPool.query('SELECT 1'))

```

Concurrent Checks

Check dependensi parallel, bukan sequential.

```

const [pgResult, redisResult, apiResult] = await Promise.all([
  checkPostgres(),

```

```

    checkRedis(),
    checkExternalAPI(),
  });

```

Graceful Shutdown

Kalo app di-terminate (SIGTERM / SIGINT), jangan langsung mati. Kasih waktu buat:

1. Berhenti nerima request baru
2. Selesaikan request yang lagi jalan
3. Tutup koneksi DB, Redis, dll.
4. Flush log

```

// src/index.ts
import express from 'express';
import logger from './lib/logger';
import { Pool } from 'pg';
import { createClient } from 'redis';

const app = express();
const server = app.listen(PORT, () => {
  logger.info({ port: PORT }, 'Server started');
});

// Graceful shutdown
async function gracefulShutdown(signal: string) {
  logger.info({ signal }, 'Received shutdown signal');

  // 1. Stop accept new connections
  server.close(() => {
    logger.info('HTTP server closed');
  });

  // 2. Wait for active requests (max 30 detik)
  const forceExit = setTimeout(() => {
    logger.error('Forced shutdown after timeout');
    process.exit(1);
  }, 30000);

  try {
    // 3. Close database connections
    await pgPool.end();
    logger.info('PostgreSQL pool closed');

    // 4. Close Redis

```

```

    await redisClient.disconnect();
    logger.info('Redis disconnected');

    // 5. Flush log
    await logger.flush?();

    clearTimeout(forceExit);
    process.exit(0);
  } catch (err) {
    logger.error({ err }, 'Error during graceful shutdown');
    process.exit(1);
  }
}

// Listen for signals
process.on('SIGTERM', () => gracefulShutdown('SIGTERM'));
process.on('SIGINT', () => gracefulShutdown('SIGINT'));

```

Connection Draining Pattern

```

sequenceDiagram
    participant K8s as Kubernetes
    participant App as App
    participant DB as Database
    participant LB as Load Balancer

    K8s->>App: SIGTERM
    App->>LB: Stop health check OK (readiness = false)
    App->>App: Wait for active requests finish
    Note over App: Selesaikan request dalam 30 detik
    App->>DB: Close connection pool
    App->>App: Flush logs
    App->>K8s: Exits with code 0

```

Monitoring Dashboard — express-status-monitor

Pasang dashboard realtime buat liat request rate, response time, status code, dll.

```
npm install express-status-monitor
```

```

// src/index.ts
import statusMonitor from 'express-status-monitor';

app.use(statusMonitor({
  title: 'Express Status', // Judul dashboard

```

```

path: '/status', // Dashboard URL
spans: [
  { interval: 1, retention: 60 }, // 1 detik, simpan 60 data
  { interval: 5, retention: 60 }, // 5 detik, simpan 60 data
  { interval: 15, retention: 60 }, // 15 detik, simpan 60 data
],
chartVisibility: {
  cpu: true,
  memory: true,
  load: true,
  eventLoop: true,
  heap: true,
  responseTime: true,
  rps: true,
  statusCodes: true,
},
healthChecks: [
  { protocol: 'http', host: 'localhost', path: '/healthz', port: PORT },
  { protocol: 'http', host: 'localhost', path: '/readyz', port: PORT },
],
}));

```

Dashboard available di: <http://localhost:3000/status>

Fitur: - **CPU & Memory usage** — realtime chart - **Request per second** — throughput - **Response time** — avg, max, min - **Status codes** — breakdown 2xx, 3xx, 4xx, 5xx - **Health check status** — hijau/merah

⚠ **Proteksi:** Dashboard expose informasi sensitif. Jangan lupa auth middleware:

```

// Proteksi dashboard middleware
app.use('/status', (req, res, next) => {
  const token = req.headers['authorization'];
  if (token !== `Bearer ${process.env.STATUS_MONITOR_TOKEN}`) {
    return res.status(401).json({ error: 'Unauthorized' });
  }
  next();
});
app.use(statusMonitor({...}));

```

K8s Probe Integration

Kalo app di-deploy ke Kubernetes, config probe-nya:

```

# deployment.yaml
apiVersion: apps/v1

```

```

kind: Deployment
spec:
  template:
    spec:
      containers:
      - name: my-app
        ports:
        - containerPort: 3000
        livenessProbe:
          httpGet:
            path: /healthz
            port: 3000
          initialDelaySeconds: 5
          periodSeconds: 10
          timeoutSeconds: 3
          failureThreshold: 3
        readinessProbe:
          httpGet:
            path: /readyz
            port: 3000
          initialDelaySeconds: 10
          periodSeconds: 5
          timeoutSeconds: 3
          failureThreshold: 2

```

Latihan

1. Buat endpoint /healthz (liveness) dan /readyz (readiness). Liveness return { status: 'ok', uptime }. Readiness check PostgreSQL (SELECT 1) + Redis (PING) dengan timeout 2 detik per check. Tulis kode lengkap + contoh response sukses dan gagal.
2. Implementasikan graceful shutdown handler: tangkap SIGTERM & SIGINT, stop HTTP server, close koneksi DB + Redis, beri waktu maksimal 20 detik. Tulis kode lengkap dengan logging tiap step. Sertakan mermaid sequence diagram.
3. Pasang express-status-monitor dashboard di /admin/status. Proteksi dashboard dengan Bearer token dari env STATUS_TOKEN. Konfigurasi health checks untuk /healthz dan /readyz. Tulis kode setup + middleware auth.
4. Buat K8s probe config (YAML) yang pake /healthz untuk liveness dan /readyz untuk readiness. Jelaskan perbedaan parameter: initialDelaySeconds, periodSeconds, timeoutSeconds,

failureThreshold untuk masing-masing probe. Kapan pake nilai agresif vs konservatif?

41.4 Production Monitoring

Uptime Monitoring

Health check endpoint aja ga cukup — kalo app mati total, health check juga ikut mati. Butuh **external monitoring** yang ngecek dari luar.

Pilihan Tools

Tool	Deployment	Harga	Fitur
Uptime Kuma	Self-hosted	Gratis	Open source, banyak notifikasi, status page
Upstash Monitor	Managed cloud	Gratis tier	Serverless, cron-based, Slack/email/webhook
UptimeRobot	Managed cloud	Free: 50 monitor	Simple, mature, banyak integrasi
Better Uptime	Managed cloud	Paid	Status page + on-call scheduling
Checkly	Managed cloud	Paid	Playwright-based, browser check

Rekomendasi

- **Personal/side project:** Uptime Kuma (self-host pake Docker) atau UptimeRobot (free tier)
- **Production tim:** Better Uptime atau Checkly (browser check beneran)

Setup Uptime Kuma

```
# Docker
docker run -d --name uptime-kuma \
  -p 3001:3001 \
  -v uptime-kuma-data:/app/data \
  louislam/uptime-kuma:latest
```

Buka <http://localhost:3001> → buat akun → add monitor:

Setting	Isi
Monitor Type	HTTP(s)
URL	https://api.example.com/healthz
Interval	60 detik
Retries	3
Notification	Slack / Discord / Email

Setup Upstash Monitor

```
# CLI
npx @upstash/monitor-cli add \
  --url https://api.example.com/healthz \
  --interval 60s \
  --notify slack
```

Generate Status Page

Buat halaman publik yang nunjukin status semua service.

```
<!-- Contoh sederhana - polling /healthz -->
<!DOCTYPE html>
<html>
<head>
  <title>System Status</title>
  <style>
    .ok { color: green; }
    .error { color: red; }
  </style>
</head>
<body>
  <h1>🔍 System Status</h1>
  <div id="checks"></div>

  <script>
    async function checkHealth() {
```

```

const services = [
  { name: 'API', url: '/healthz' },
  { name: 'Database', url: '/readyz' },
];

const checksEl = document.getElementById('checks');
checksEl.innerHTML = '';

for (const svc of services) {
  try {
    const res = await fetch(svc.url);
    const data = await res.json();
    checksEl.innerHTML += `<p class="${data.status === 'ok' ? 'ok' : 'error'}" >
      ${data.status === 'ok' ? '✅' : '❌'} ${svc.name}: ${data.status}
    </p>`;
  } catch {
    checksEl.innerHTML += `<p class="error">❌ ${svc.name}: unreachable</p>`;
  }
}

checkHealth();
setInterval(checkHealth, 30000);
</script>
</body>
</html>

```

Alerting

Bukan cuma tau ada error — **harus dikasih tau** pas terjadi.

Alert Channels

Channel	Cocok buat	Latency
Email	Non-urgent, report harian	1-5 menit
Slack webhook	Team notification	~detik
Discord webhook	Team notification	~detik
PagerDuty/Opsgenie	On-call, urgent	~detik
SMS/Phone	Critical, 24/7	~menit

Slack Webhook

```

// src/lib/alert.ts
import logger from './logger';

```

```

const SLACK_WEBHOOK_URL = process.env.SLACK_WEBHOOK_URL;

interface AlertPayload {
  title: string;
  message: string;
  severity: 'critical' | 'warning' | 'info';
  service: string;
  timestamp: string;
  metadata?: Record<string, unknown>;
}

export async function sendSlackAlert(payload: AlertPayload) {
  if (!SLACK_WEBHOOK_URL) {
    logger.warn('SLACK_WEBHOOK_URL not set, skipping alert');
    return;
  }

  const color = payload.severity === 'critical' ? '#FF0000'
    : payload.severity === 'warning' ? '#FFA500'
    : '#3498DB';

  const blocks = [
    {
      type: 'header',
      text: { type: 'plain_text', text: `🔔 ${payload.title}` },
    },
    {
      type: 'section',
      fields: [
        { type: 'mrkdwn', text: `*Service:* \`${payload.service}\` ` },
        { type: 'mrkdwn', text: `*Severity:* \`${payload.severity}\` ` },
        { type: 'mrkdwn', text: `*Time:* \`${payload.timestamp}\` ` },
      ],
    },
    { type: 'section', text: { type: 'mrkdwn', text: payload.message } },
  ];

  if (payload.metadata) {
    blocks.push({
      type: 'section',
      fields: Object.entries(payload.metadata).map(([k, v]) => ({
        type: 'mrkdwn',
        text: `*${k}:* \`${v}\` ` ,
      })),
    });
  }
}

```

```

    }

    try {
      await fetch(SLACK_WEBHOOK_URL, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ attachments: [{ color, blocks }] }),
      });
      logger.info({ severity: payload.severity }, 'Alert sent to Slack');
    } catch (err) {
      logger.error({ err }, 'Failed to send Slack alert');
    }
  }
}

```

Discord Webhook

```

export async function sendDiscordAlert(payload: AlertPayload) {
  const webhookUrl = process.env.DISCORD_WEBHOOK_URL;
  if (!webhookUrl) return;

  const color = payload.severity === 'critical' ? 0xFF0000
    : payload.severity === 'warning' ? 0xFFA500
    : 0x3498DB;

  const embed = {
    title: payload.title,
    description: payload.message,
    color,
    fields: [
      { name: 'Service', value: payload.service, inline: true },
      { name: 'Severity', value: payload.severity, inline: true },
      { name: 'Time', value: payload.timestamp, inline: true },
    ],
    timestamp: payload.timestamp,
  };

  try {
    await fetch(webhookUrl, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ embeds: [embed] }),
    });
  } catch (err) {
    logger.error({ err }, 'Failed to send Discord alert');
  }
}

```

```
}
```

Alert Threshold Logic

```
// src/lib/alertManager.ts
import logger from './logger';
import { sendSlackAlert } from './alert';

class AlertManager {
  private errorCounts = new Map<string, { count: number; windowStart: number }>();
  private readonly windowMs = 5 * 60 * 1000; // 5 menit
  private readonly threshold = 10; // 10 error dalam 5 menit

  recordError(service: string, errorMessage: string) {
    const now = Date.now();
    const key = service;

    let record = this.errorCounts.get(key);
    if (!record || now - record.windowStart > this.windowMs) {
      record = { count: 0, windowStart: now };
    }

    record.count++;
    this.errorCounts.set(key, record);

    if (record.count >= this.threshold) {
      sendSlackAlert({
        title: 'Error Rate Threshold Breached',
        message: `${service} has ${record.count} errors in 5 minutes. Last: ${errorMessage}`,
        severity: 'critical',
        service,
        timestamp: new Date().toISOString(),
        metadata: { errorCount: record.count, windowMinutes: this.windowMs / 60000 },
      });

      // Reset counter
      this.errorCounts.set(key, { count: 0, windowStart: now });
    }
  }
}

export const alertManager = new AlertManager();

// Pake:
app.use((err, req, res, next) => {
```

```

    alertManager.recordError('api', err.message);
    next(err);
  });

```

APM — Application Performance Monitoring

APM = ngeliat **kenapa** aplikasi lambat, bukan cuma tau kalo lambat.

Components

```

graph LR
    A[User Request] --> B[Load Balancer]
    B --> C[Express App]
    C --> D[DB Query]
    C --> E[External API]
    C --> F[Cache]

    subgraph APM
        G[Response Time Waterfall]
        H[DB Profiling]
        I[Memory Profiling]
        J[Error Tracking]
    end

    end

    C --> G
    D --> H
    C --> I
    C --> J

```

Response Time Waterfall

Breakdown waktu tiap span dalam satu request:

```

GET /api/orders/123 → total: 850ms
├── Middleware          15ms
├── Auth check          22ms
├── DB query (orders)  320ms ← bottleneck
├── DB query (items)  280ms ← bottleneck
├── External API call  180ms
└── Response serialize 33ms

```

Cara baca: - Cari span paling panjang → itu bottleneck - Bandingin p50 vs p99 — kalo p99 jauh lebih besar, ada request yang outlier - DB query lambat? Tambah index atau cache - External API lambat? Set timeout lebih ketat atau pake circuit breaker

DB Query Profiling

Jangan tebak — ukur query mana yang lambat.

```
// Prisma middleware — log query lambat
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

prisma.$use(async (params, next) => {
  const start = Date.now();
  const result = await next(params);
  const duration = Date.now() - start;

  if (duration > 100) {
    // Log query yang lambat (>100ms)
    logger.warn({
      model: params.model,
      action: params.action,
      duration: `${duration}ms`,
      args: JSON.stringify(params.args).slice(0, 200),
    }, 'Slow database query');
  }

  return result;
});
```

Memory Leak Detection

Memory leak = memory usage naek terus ga pernah turun.

```
// Memory usage monitoring
function logMemoryUsage() {
  const used = process.memoryUsage();
  logger.info({
    rss: `${Math.round(used.rss / 1024 / 1024)}MB`,
    heapTotal: `${Math.round(used.heapTotal / 1024 / 1024)}MB`,
    heapUsed: `${Math.round(used.heapUsed / 1024 / 1024)}MB`,
    external: `${Math.round(used.external / 1024 / 1024)}MB`,
  }, 'Memory usage');
}

// Log setiap 5 menit
setInterval(logMemoryUsage, 5 * 60 * 1000);

// Alert kalo heapUsed > 500MB
setInterval(() => {
```



```

const heapUsedMB = process.memoryUsage().heapUsed / 1024 / 1024;
if (heapUsedMB > 500) {
  sendSlackAlert({
    title: 'High Memory Usage',
    message: `Heap usage at ${Math.round(heapUsedMB)}MB`,
    severity: 'warning',
    service: 'api',
    timestamp: new Date().toISOString(),
    metadata: { heapUsed: `${Math.round(heapUsedMB)}MB` },
  });
}
}, 60000);

```

Common APM Tools

Tool	Deployment	Fitur Utama
Sentry Performance	Managed	Tracing + error dalam satu platform
Datadog APM	Managed	Waterfall, dashboard, integration 700+
New Relic	Managed	Distributed tracing, AI insights
OpenTelemetry	Self-hosted	Open standard, vendor-agnostic
Grafana + Tempo	Self-hosted	Traces + metrics + logs satu atap

Incident Response Checklist

Kalo production down — jangan panik. Ikutin checklist.

❑ 1. Detect & Triage

- ❑ Apakah benar ini incident? (bukan false alarm)
- ❑ Service mana yang kena? (API, DB, Worker, dll)
- ❑ Severity level? (critical / major / minor)
- ❑ Berapa impact? (semua user? fitur tertentu?)

Severity Levels:

Level	Definisi	Response Time
P0 — Critical	Semua user ga bisa akses, data loss	< 15 menit

Level	Definisi	Response Time
P1 — Major	Fitur utama broken, sebagian user	< 1 jam
P2 — Minor	Fitur non-krusial broken, ga ada workaround	< 4 jam
P3 — Low	Bug kecil, ada workaround	Next sprint

□ 2. Communicate

- Notify team via Slack/Discord (buat channel #incident)
- Update status page (jika ada)
- Assign incident lead
- Start incident channel / thread

Template pesan:

□ INCIDENT: [TITLE]
 Service: API
 Severity: P1
 Start time: 14:30 UTC
 Impact: Order creation broken for all users
 Status: INVESTIGATING
 Lead: @midory

□ 3. Investigate

- Cek Sentry – error baru? release mana?
- Cek logs – pake grep / jq cari pola error
- Cek metrics – CPU, memory, request rate spike?
- Cek DB – slow queries, connections, replication lag
- Cek external API – apakah upstream down?
- Cek recent deploy / config change
- Rollback if needed – kembalikan ke release sebelumnya

□ 4. Mitigate & Resolve

- Apply hotfix / rollback
- Restart service jika perlu
- Scale up jika traffic spike
- Clear cache jika data corrupt
- Verify fix – test endpoint health
- Monitor after fix – liat sentry, logs, metrics 15 menit

□ 5. Follow Up

- Update status page → RESOLVED
- Write postmortem dalam 24 jam
 - Timeline (detect → investigate → fix → resolve)
 - Root cause analysis (5 Whys)
 - Action items (prevent recurrence)
 - What went well / what went wrong
- Schedule action items ke sprint berikutnya
- Share postmortem ke team

Template Postmortem

Postmortem: [Title]

Date: 2025-03-20

Severity: P1

Duration: 47 minutes (14:30 – 15:17 UTC)

Impact: 12,000 failed orders, ~\$3,200 revenue loss

Timeline

Time	Event
-----	-----
14:30	PagerDuty alert: 5xx error rate spike
14:32	Sentry: 1000+ errors in 2 min
14:35	Investigation started
14:40	Found: DB connection pool exhausted
14:45	Rolled back deploy to v1.2.0
15:00	DB pool recovered, errors dropping
15:10	All endpoints healthy
15:17	Status page → RESOLVED

Root Cause

Deploy v1.2.3 introduced a bug where `db.query()` inside a `forEach` loop created 50 concurrent connections per request instead of reusing pool.

Action Items

- [] Add DB pool max connection alert in Sentry
- [] Add connection pool monitoring dashboard
- [] Code review: forbid `db.query()` inside loops
- [] Staged rollout: 10% → 50% → 100%

Latihan

1. Setup Uptime Kuma via Docker (atau konfigurasi UptimeRobot) untuk monitor endpoint `/healthz` tiap 60 detik. Konfigurasi no-

tifikasi ke Slack/Discord kalo down. Tulis docker-compose.yml + screenshot konfigurasi monitor.

2. Buat fungsi `sendSlackAlert` dan `sendDiscordAlert` yang kirim pesan terformat (title, severity, message, timestamp, metadata). Tulis kode lengkap + payload JSON contoh untuk severity critical dan warning.
3. Implementasikan `AlertManager`: hitung error rate per service dalam window 5 menit, kirim alert kalo threshold 10 error terlewati. Reset counter setelah alert. Integrasikan ke Express error handler. Tulis kode lengkap.
4. Buat memory monitoring: log memory usage (rss, heapTotal, heapUsed) tiap 5 menit, kirim alert kalo heapUsed > 400MB. Buat DB query profiler untuk Prisma/ORM — log query yang > 200ms. Tulis kode lengkap.